



Daniel Macharia <danielmacharia43@gmail.com>

YOHPAL LIVE v2 — FINAL MASTER DEVELOPER HANDOVER MANUAL

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Fri, 29 May at 13:02

YOHPAL LIVE v2 — FINAL MASTER DEVELOPER HANDOVER MANUAL

Version

v2.9 — Final Unified Production Handover

1. Executive Overview

YohPal Live v2 is a unified AI-driven short-video, livestreaming, monetisation, creator-economy, chat, ads, and social discovery platform.

This manual is the single operational document for:

- setup
- architecture understanding
- merge sequencing
- implementation standards
- deployment
- scaling
- production hardening
- QA
- launch readiness

This document supersedes scattered implementation chats.

2. Product Vision

YohPal Live v2 is designed to become:

- the AI-first alternative to TikTok
- the monetisation center for African creators
- the livestream commerce engine for the YohPal ecosystem
- the realtime media backbone for YohPal + ICS ecosystem
- the creator growth and affiliate acquisition engine
- the future social discovery/search platform

Core differentiators:

- AI Creator Studio
 - lower-quarter livestream ads every 15 seconds
 - own RTC infrastructure (not ZEGOCLOUD)
 - creator monetisation from day one
 - Infinity Polls
 - hybrid AI search
 - affiliate-powered viral growth
 - WebRTC + RTMP/HLS hybrid live architecture
 - backend-only wallet security
 - sequence-first realtime chat architecture
-

3. Master Repository Structure

```
/apps
/mobile_flutter
/wallet_web
/admin_web

/backend
/firebase_functions
/cloud_run_workers
```

```
/rtc_media_server
/shared_contracts

/firestore
/infra
/scripts
/docs
/.github
```

4. Official Technology Stack

Mobile

- Flutter
- Firebase Auth
- Firestore
- Firebase Messaging
- video_player
- flutter_webrtc
- provider

Backend

- Firebase Functions
- TypeScript
- Node.js
- Redis
- PostgreSQL (future scale layer)
- Socket.IO
- Mediasoup

Workers

- Cloud Run
- Python 3.11
- FastAPI
- FFmpeg
- OpenAI / Vertex AI

Web

- Next.js 14
- React
- Firebase Admin SDK

Media

- WebRTC
- RTMP
- HLS
- FFmpeg
- Coturn

Search

- Firestore
 - OpenSearch
 - AI intent layer
-

5. Golden Engineering Rules

Rule 1 — Flutter Cannot Be Trusted

Flutter may:

- request actions
- create commands
- create wallet sessions
- display state

Flutter may NOT:

- complete wallet payments
- write walletLedger
- assign chat sequence numbers
- write creator earnings
- approve payouts
- write ad spend
- write AI results

Rule 2 — Shared Contracts Are Source of Truth

All DTOs/contracts live in:

/backend/shared_contracts

No engineer should invent fields outside shared contracts.

Rule 3 — Event-Driven Architecture

Important actions must flow through:

```
client
→ command
→ backend validation
→ event
→ persistence
→ notification
→ analytics
```

Rule 4 — Sequence-Based Chat

Realtime delivery is NOT source of truth.

Source of truth:

```
sequence numbers
+ sync
+ idempotency
```

6. Step-by-Step Merge Order

Step 1

Merge:

Final Unified Run-to-Green Production Repo Pack

This establishes:

- repo structure
 - Firebase
 - workers
 - RTC shell
 - wallet web shell
 - admin web shell
-

Step 2

Merge overlays in exact order:

1. Step 2.1 — Module completion
2. Step 2.2 — Video feed
3. Step 2.3 — AI Creator Studio
4. Step 2.4 — Live + multistreaming
5. Step 2.5 — Monetisation + wallet
6. Step 2.6 — Ads + polls + search
7. Step 2.7 — Chat + social graph
8. Step 2.8 — Final hardening

Do NOT merge randomly.

7. Environment Setup

Required Tools

Install:

- Flutter 3.22+
 - Node.js 20+
 - Firebase CLI
 - Docker
 - Python 3.11
 - FFmpeg
 - Git
 - Google Cloud SDK
-

Clone Repo

```
git clone <repo>
cd yohpal-live-v2
```

Environment File

Create:

```
.env
```

Populate:

```
FIREBASE_PROJECT_ID=
FIREBASE_STORAGE_BUCKET=
CDN_BASE_URL=
OPENAI_API_KEY=
MPESA_CONSUMER_KEY=
MPESA_CONSUMER_SECRET=
MPESA_SHORTCODE=
MPESA_PASSKEY=
PUBLIC_IP=
TURN_URL=
TURN_USERNAME=
TURN_CREDENTIAL=
REDIS_URL=
```

8. Firebase Setup

Login

```
firebase login
```

Initialize

```
firebase use <project-id>
```

Deploy Rules + Indexes

```
firebase deploy --only firestore:rules
firebase deploy --only firestore:indexes
```

Emulator

```
firebase emulators:start --only firestore,functions
```

9. Flutter Mobile Setup

Install Packages

```
cd apps/mobile_flutter
flutter pub get
```

Run Analyze

```
flutter analyze
```

Run Tests

```
flutter test
```

Start App

```
flutter run
```

10. Firebase Functions Setup

Install

```
cd backend/firebase_functions
npm install
```

Build

```
npm run build
```

Serve Locally

```
firebase emulators:start --only functions,firestore
```

11. Cloud Run Workers Setup

Video Transcoder

```
cd backend/cloud_run_workers/video_transcoder
pip install -r requirements.txt
uvicorn main:app --reload --port 8080
```

AI Video Worker

```
cd backend/cloud_run_workers/ai_video_worker
pip install -r requirements.txt
uvicorn main:app --reload --port 8081
```

Multistream Worker

```
cd backend/cloud_run_workers/multistream_worker
pip install -r requirements.txt
uvicorn main:app --reload --port 8082
```

Search Indexer

```
cd backend/cloud_run_workers/search_indexer
pip install -r requirements.txt
uvicorn main:app --reload --port 8083
```

12. RTC Media Engine Setup

Install

```
cd backend/rtc_media_server
npm install
```

Build

```
npm run build
```

Run

```
npm run dev
```

Docker Compose

```
cd infra/docker
docker compose -f docker-compose.full-local.yml up --build
```

13. WebRTC Production Checklist

Infrastructure

- static public IP
- UDP ports open
- Coturn configured
- TLS enabled
- Redis configured
- FFmpeg installed

Mediasoup

- worker starts
- transports connect
- producers created
- consumers created
- disconnect cleanup works

Viewer Experience

- viewer joins within 3 seconds
 - reconnect works
 - viewer count updates
 - livestream can end safely
-

14. Video Feed Architecture

Playback Flow

```
Firestore videos
→ repository
→ controller
→ predictive prefetch
→ controller pool
→ video_player
```

Feed Rules

- only live videos render
 - broken videos auto-skip
 - HLS required
 - first 5 videos prewarm
 - aggressive scroll expands prefetch window
 - old controllers disposed automatically
-

15. AI Creator Studio

Features

- captions
 - hooks
 - hashtags
 - translations
 - trim suggestions
 - AI thumbnails
 - viral score
-

Processing Flow

creator action
→ aiVideoJob
→ Cloud Run worker
→ AI generation
→ [video.ai](#) updated
→ Flutter UI updates

16. Livestreaming Architecture

Hybrid Strategy

WebRTC

Use for:

- interactive streams
- classrooms
- co-hosting
- realtime conversations

RTMP/HLS

Use for:

- large audiences
 - replay
 - multistreaming
-

Lower-Quarter Live Ads

Ads appear every:

15 seconds

Meaning:

4 ads per minute

without interrupting livestream.

This is a major monetisation differentiator.

17. Monetisation Architecture

Supported Revenue

- gifts
- tips
- subscriptions
- boosts
- ad revenue share
- affiliate commissions
- creator earnings

- AI tools
 - livestream commerce
-

Wallet Flow

Flutter
→ walletSession
→ web wallet
→ M-Pesa callback
→ backend verification
→ walletLedger
→ earnings update

Security Rules

Collections backend-only:

- walletLedger
 - creatorEarnings
 - adRevenueShare
 - affiliateCommissions
 - riskReviews
-

18. Ads Architecture

Ad Placements

- in-feed
 - livestream lower-quarter
 - sponsored polls
 - boosted videos
 - search ads
-

Revenue Split

Suggested baseline:

Creator = 60%
Platform = 40%

19. Infinity Polls

Poll Types

- live polls
 - video polls
 - Infinity Polls
 - prediction polls
 - sponsored polls
-

Fraud Protection

- deterministic vote IDs
 - idempotency
 - device checks
 - account age scoring
 - backend-only totals
-

20. Search Architecture

Hybrid Search

Firestore

- + OpenSearch
 - + AI intent layer
-

Search Targets

- videos
 - creators
 - livestreams
 - polls
 - hashtags
-

21. Chat Architecture

Core Principle

Messages are sequence-based.

```
command
→ backend assigns seq
→ message persisted
→ notifications
→ SDK sync
```

Features

- DMs
 - group chat
 - live chat
 - replies
 - reactions
 - edits
 - soft delete
 - AI assistant
-

22. Notifications

Notification Types

- new message
 - creator live
 - poll result
 - wallet update
 - gift received
 - follower update
-

Delivery

Use:

- Firebase Messaging
 - notification documents
 - deep links
-

23. Social Graph

Supported Edges

- follows
 - subscriptions
 - fans
 - blocks
-

Growth Logic

YohPal Live growth depends on:

- creator fan migration
 - contacts import
 - affiliate referrals
 - social share links
 - WhatsApp virality
-

24. CI/CD

GitHub Actions

Pipelines:

- Flutter analyze/test
 - Firebase Functions build
 - RTC server build
 - wallet web build
 - admin web build
-

Deploy Scripts

Firebase

```
bash scripts/firebase_deploy.sh
```

Cloud Run

```
bash scripts/cloud_run_deploy.sh
```

RTC

```
bash scripts/rtc_deploy_docker.sh
```

25. Emulator Security Tests

Wallet

- client cannot write walletLedger
- client cannot write creatorEarnings

Chat

- client cannot write chatMessages
- only chatCommands writable

Polls

- duplicate vote blocked

Ads

- advertiser cannot directly increase spend
-

26. Production Scaling Strategy

Phase 1

- Firebase
- Firestore
- Functions
- Cloud Run

- single RTC node

Phase 2

- Redis room registry
- Socket sticky sessions
- OpenSearch cluster
- multi-worker media processing

Phase 3

- PostgreSQL message persistence
 - Redis sequence engine
 - Redis Pub/Sub EventBus
 - Kubernetes orchestration
-

27. Monitoring

Track:

- livestream uptime
 - FFmpeg failures
 - RTC failures
 - wallet callback failures
 - payout failures
 - Cloud Run errors
 - video playback failures
 - AI worker failures
 - poll fraud spikes
 - ad delivery health
-

28. Launch Strategy

Closed Alpha

- internal team
- selected creators
- limited livestreaming
- limited payouts

Closed Beta

- campus creators
- livestream commerce
- affiliate campaigns
- social growth testing

Public Launch

Only after:

- wallet stability
 - livestream stability
 - moderation readiness
 - fraud controls
 - payout reconciliation
-

29. Final Non-Negotiables

Never:

- bypass shared contracts
- bypass backend wallet processing
- bypass chat sequencing
- bypass Firestore rules
- hardcode production secrets
- merge random schema changes

Always:

- run emulator tests
 - run CI before merge
 - deploy rules before features
 - use idempotency for payments
 - use backend-only ledger writes
-

30. Final Signoff Checklist

Engineering

- Flutter stable
- Functions stable
- Workers stable
- RTC stable
- Web apps stable

Security

- Firestore rules deployed
- Wallet protected
- Chat protected
- Ads protected

Product

- Video feed stable
- AI Creator Studio stable
- Livestreaming stable
- Monetisation stable
- Ads stable
- Polls stable
- Chat stable
- Search stable

Go/No-Go

Approved by:

Date:

Notes: